

TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA Công Nghệ Thông Tin
BỘ MÔN: MẠNG & TT

ĐỀ THI VÀ BÀI LÀM - Đề 1

Tên học phần: Lập Trình Mạng

Mã học phần: Hình thức thi: *Tự luận có giám sát*

Thời gian làm bài: 30 phút (*không kể thời gian phát đề và nộp bài*)

Được sử dụng tài liệu khi làm bài. Không chia sẻ bài cho nhau, nếu phát hiện sẽ chia đều số điểm.

Họ tên: Nguyễn Hoàng Phúc **Lớp:** 23T_DT1 **MSSV:** 102230207

Sinh viên làm bài trực tiếp trên tệp này, lưu tệp với định dạng MSSV_HọTên.pdf và nộp bài thông qua MSTeam:

Câu 1 (8 điểm): Hãy xây dựng chương trình chia sẻ màn hình từ server cho các client. Lưu ý cần tối ưu cho mỗi client về cả đường truyền lẫn chất lượng hình ảnh.

Trả lời:

Dán code server vào bên dưới

```
package sv102230207_NguyenHoangPhuc;

import java.awt.Rectangle;
import java.awt.Robot;
import java.awt.Toolkit;
import java.awt.image.BufferedImage;
import java.io.BufferedOutputStream;
import java.io.ByteArrayOutputStream;
import java.io.DataOutputStream;
import java.net.ServerSocket;
import java.net.Socket;

import javax.imageio.IIOImage;
import javax.imageio.ImageIO;
import javax.imageio.ImageWriteParam;
import javax.imageio.ImageWriter;
import javax.imageio.stream.ImageOutputStream;

public class ScreenServer1 {
    public static void main(String[] args) {
        new ScreenServer1();
    }

    public ScreenServer1() {
        Screen s = new Screen();
        s.start();
        try {
            ServerSocket server = new ServerSocket(2345);
            System.out.println("Server listening on port 2345...");
            while (true) {
                Socket soc = server.accept();
                ScreenProcessing sp = new ScreenProcessing(soc);
                sp.start();
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

```

        System.out.println("Client connected: " +
soc.getInetAddress().getHostAddress());
    }
    } catch (Exception e) {
        e.printStackTrace();
    }
}

class Screen extends Thread {
    static volatile byte[] tmp; // volatile để thread khác thấy cập nhật
mới
    static volatile int count = 0;

    @Override
    public void run() {
        try {
            Robot r = new Robot();
            Rectangle capture = new
Rectangle(Toolkit.getDefaultToolkit().getScreenSize());

            long startTime = System.currentTimeMillis();
            while (true) {
                BufferedImage img = r.createScreenCapture(capture);

                ByteArrayOutputStream bos = new
ByteArrayOutputStream();
                ImageWriter jpgWriter =
ImageIO.getImageWritersByFormatName("jpg").next();
                ImageWriteParam jpgWriteParam =
jpgWriter.getDefaultWriteParam();

                jpgWriteParam.setCompressionMode(ImageWriteParam.MODE_EXPLICIT);
                jpgWriteParam.setCompressionQuality(0.5f);

                ImageOutputStream ios =
ImageIO.createImageOutputStream(bos);
                jpgWriter.setOutput(ios);
                jpgWriter.write(null, new IIOMImage(img, null, null),
jpgWriteParam);

                ios.close();
                jpgWriter.dispose();

                tmp = bos.toByteArray();
                count++;

                // log tốc độ capture (server side)
                if (count % 100 == 0) {
                    long now = System.currentTimeMillis();
                    long elapsed = now - startTime;
                    System.out.println("Captured 100 frames in " +
elapsed + " ms");

                    startTime = now; // reset lại mốc
                }
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

```

    }
}

class ScreenProcessing extends Thread {
    Socket soc;
    DataOutputStream output;
    int countNow;

    public ScreenProcessing(Socket soc) {
        try {
            this.soc = soc;
            this.output = new DataOutputStream(new
BufferedOutputStream(soc.getOutputStream()));
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    public void run() {
        int frameSent = 0;
        long startTime = System.currentTimeMillis();

        while (true) {
            try {
                if (countNow == Screen.count) {
                    Thread.sleep(5);
                    continue;
                }
                byte[] tmp = Screen.tmp;
                countNow = Screen.count;

                output.writeInt(tmp.length);
                output.write(tmp);
                output.flush();

                frameSent++;
                if (frameSent % 100 == 0) {
                    long now = System.currentTimeMillis();
                    long elapsed = now - startTime;
                    System.out.println("Client: " +
soc.getInetAddress().getHostAddress() + " | Sent 100 frames in "
                    + elapsed + " ms");
                    startTime = now; // reset lại mốc
                }

            } catch (Exception e) {
                System.out.println("Client disconnected: " +
soc.getInetAddress().getHostAddress());
                break;
            }
        }
    }
}

```

Dán code client vào bên dưới

```

package sv102230207_NguyenHoangPhuc;

import java.awt.Graphics;

```

```

import java.awt.Image;
import java.awt.image.BufferedImage;
import java.io.ByteArrayInputStream;
import java.io.DataInputStream;
import java.net.Socket;
import javax.imageio.ImageIO;
import javax.swing.*;

public class ScreenClient1 extends JFrame {
    Socket soc;
    BufferedImage currentImage = null;
    ScreenPanel panel;

    public static void main(String[] args) {
        new ScreenClient1();
    }

    public ScreenClient1() {
        this.setTitle("Share Screen");
        this.setSize(800, 600);
        this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        panel = new ScreenPanel();
        this.add(panel);

        try {
            soc = new Socket("localhost", 2345);
            System.out.println("Connected to server...");

            new Thread(this::receiveFrames).start();
        } catch (Exception e) {
            e.printStackTrace();
            System.exit(1);
        }

        this.setVisible(true);
    }

    private void receiveFrames() {
        try {
            DataInputStream bis = new DataInputStream(soc.getInputStream());

            int frameCount = 0;
            long startTime = System.currentTimeMillis();
            while (true) {
                int n = bis.readInt();
                byte tmp[] = bis.readNBytes(n);

                ByteArrayInputStream bis1 = new ByteArrayInputStream(tmp);
                currentImage = ImageIO.read(bis1);

                frameCount++;
                if (frameCount % 100 == 0) {
                    long now = System.currentTimeMillis();
                    long elapsed = now - startTime;
                    System.out.println("Received 100 frames in " + elapsed +
" ms");
                    startTime = now; // reset mốc
                }
            }
        }
    }
}

```

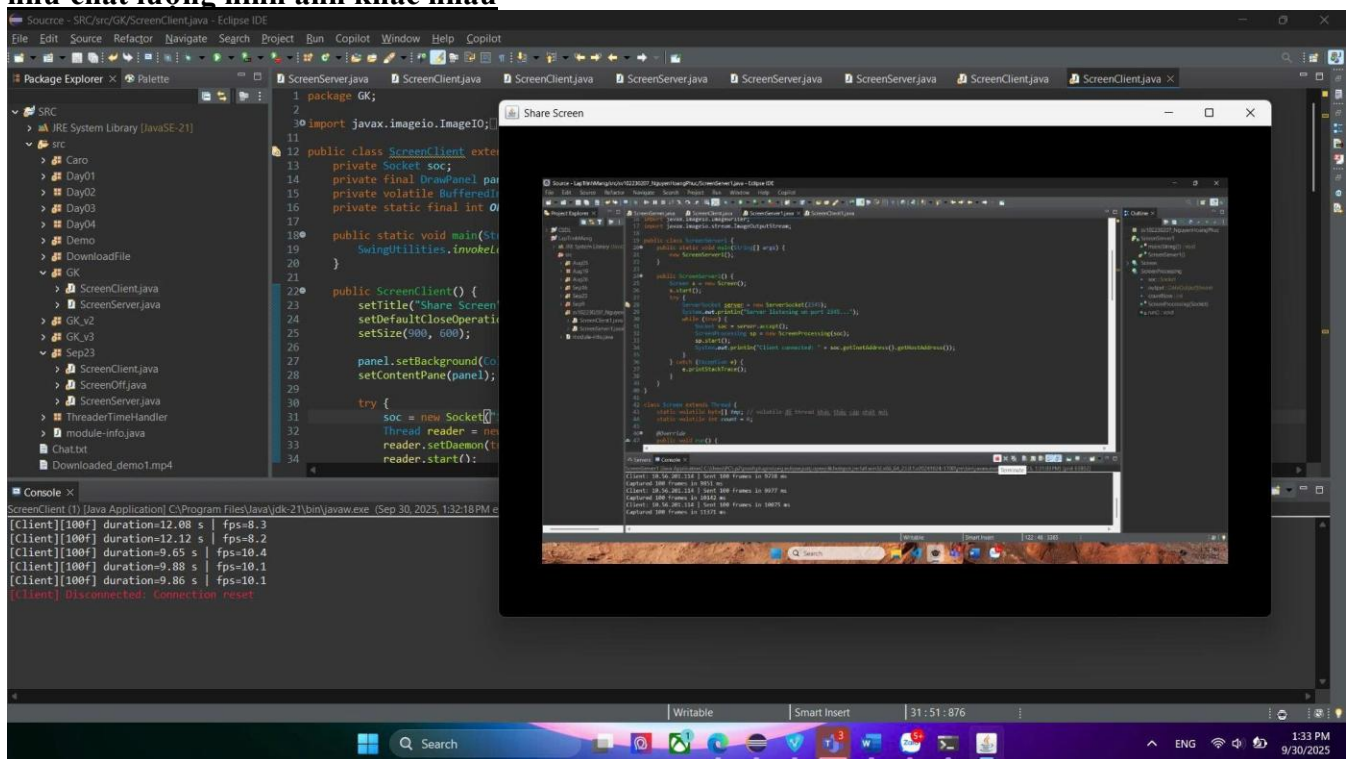
```

        panel.repaint(); // chỉ vẽ lại panel
    }
} catch (Exception e) {
    e.printStackTrace();
}
}

class ScreenPanel extends JPanel {
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        if (currentImage != null) {
            int w = this.getWidth();
            int h = this.getHeight();
            Image img2 = currentImage.getScaledInstance(w, h,
Image.SCALE_SMOOTH);
            g.drawImage(img2, 0, 0, null);
        }
    }
}
}
}

```

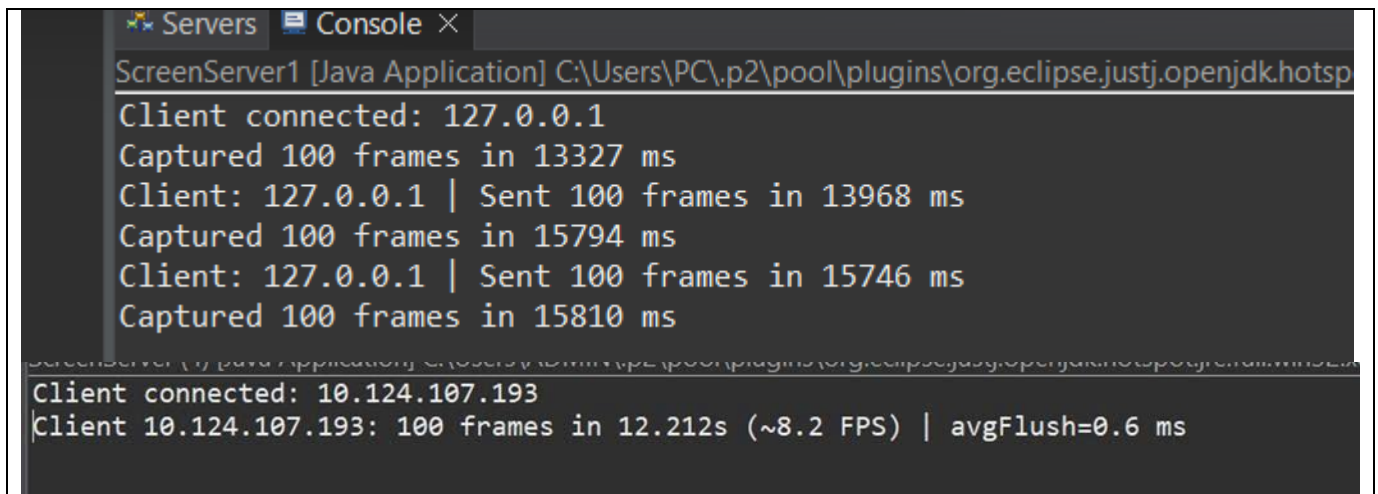
Dán một số các kết quả thực thi vào bên dưới cho thấy mỗi client có tốc độ đường truyền cũng như chất lượng hình ảnh khác nhau



```

ScreenServer1 [Java Application] C:\Users\PC\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_23.0.1.v20241024-1700\jre\bin\javaw.exe (Sep 30, 2025, 1:34:20 PM) [p
Client connected: 10.124.107.23
Client disconnected: 10.124.107.23
Captured 100 frames in 11111 ms
Client: 10.124.107.23 | Sent 100 frames in 10538 ms
Captured 100 frames in 10218 ms
Client: 10.124.107.23 | Sent 100 frames in 10080 ms

```

The screenshot shows the Eclipse IDE's console window with two tabs: 'Servers' and 'Console'. The 'Console' tab is active, displaying the output of a Java application. The output shows three separate cycles of frame capture and transmission. Each cycle starts with 'Client connected: 127.0.0.1', followed by 'Captured 100 frames in [time] ms', then 'Client: 127.0.0.1 | Sent 100 frames in [time] ms', and finally 'Captured 100 frames in [time] ms'. The times for capturing and sending frames are very close, ranging from approximately 13,327 ms to 15,810 ms. Below this, a new client connects from '10.124.107.193', and the output shows 'Client 10.124.107.193: 100 frames in 12.212s (~8.2 FPS) | avgFlush=0.6 ms'.

```
Servers Console X
ScreenServer1 [Java Application] C:\Users\PC\p2\pool\plugins\org.eclipse.justj.openjdk.hotsp
Client connected: 127.0.0.1
Captured 100 frames in 13327 ms
Client: 127.0.0.1 | Sent 100 frames in 13968 ms
Captured 100 frames in 15794 ms
Client: 127.0.0.1 | Sent 100 frames in 15746 ms
Captured 100 frames in 15810 ms
Client connected: 10.124.107.193
Client 10.124.107.193: 100 frames in 12.212s (~8.2 FPS) | avgFlush=0.6 ms
```

Câu 2 (2 điểm): Hãy mô tả bằng lời rằng em đã làm gì trong đoạn code ở trên để tối ưu về tốc độ đường truyền, chất lượng hình ảnh cho mỗi client. **Bạn nào dùng AI để trả lời coi như không có điểm.**

Trả lời:

- Để tối ưu về tốc độ đường truyền, chất lượng hình ảnh cho mỗi client em đã:

+ Chuyển ảnh từ PNG sang JPG với chất lượng 50% ảnh (dung lượng của ảnh nhẹ hơn) sau khi Robot chụp sẽ nén giúp tốc độ truyền ảnh qua cho client nhanh hơn so với ảnh PNG.

```
BufferedImage img = r.createScreenCapture(capture);
```

```
ByteArrayOutputStream bos = new ByteArrayOutputStream();
ImageWriter jpgWriter = ImageIO.getImageWritersByFormatName("jpg").next();
ImageWriteParam jpgWriteParam = jpgWriter.getDefaultWriteParam();
jpgWriteParam.setCompressionMode(ImageWriteParam.MODE_EXPLICIT);
jpgWriteParam.setCompressionQuality(0.5f);
```

+ Khi có một client kết nối đến server (server.accept()), sẽ tạo một đối tượng ScreenProcessing mới.

+ ScreenProcessing kế thừa từ Thread, khi gọi sp.start(), sẽ có một luồng riêng biệt để cho mỗi client kết nối với server.

```
+ Trong Client, new Thread(this::receiveFrames).start();
```

Thay vì thực thi trực tiếp trên luồng giao diện UI, lệnh này tạo một Thread mới và chạy hàm receiveFrames(), quá trình này đọc dữ liệu qua Socket không ảnh hưởng đến giao diện Swing.

+ Sử dụng từ khoá volatile đảm bảo các thread nhìn thấy giá trị mới nhất khi kết nối đến server, giúp các luồng gửi dữ liệu cho client không bị đọc dữ liệu cũ.